

หน่วยที่ 3

หลักการเขียนโปรแกรม เชิงวัตถุเบื้องต้น



วัตถุประสงค์เชิงพฤติกรรม



- 1.อธิบายชนิดข้อมูล ตัวแปร และค่าคงที่ในภาษา Visual C#
- 2.อธิบายและเลือกใช้ตัวดำเนินการ (Operator) ประเภทต่าง ๆ
- 3.อธิบายโครงสร้างพื้นฐานของโปรแกรม และเขียนฟังก์ชันแบบลำดับ
- 4.เขียนโปรแกรมแบบลำดับเบื้องต้นโดยใช้ฟังก์ชันสำเร็จรูป



หน่วยที่ 3 หลักการเขียนโปรแกรมเชิงวัตถุเบื้องต้น

วัตถุประสงค์เชิงพฤติกรรม

1. อธิบายชนิดข้อมูล ตัวแปร และค่าคงที่ในภาษา Visual C#
2. อธิบายและเลือกใช้ตัวดำเนินการ (Operator) ประเภทต่าง ๆ
3. อธิบายโครงสร้างพื้นฐานของโปรแกรม และเขียนผังงานแบบลำดับ
4. เขียนโปรแกรมแบบลำดับเบื้องต้นโดยใช้ฟังก์ชันสำเร็จรูป

3.1 ความรู้พื้นฐานเกี่ยวกับภาษา Visual C#

3.1.1 ชนิดข้อมูล (Data Type)

ในภาษาซีชาร์ปได้มีการกำหนดชนิดของข้อมูลไว้หลากหลายชนิด เพื่อให้สามารถรองรับข้อมูลได้หลายประเภทตามลักษณะการใช้งาน โดยแบ่งออกเป็น 4 กลุ่มหลัก คือ กลุ่มจำนวนเต็ม กลุ่มจำนวนทศนิยม กลุ่มตรรกะ และกลุ่มตัวอักษร/ข้อความ

ตารางชนิดข้อมูลทั้ง 13 ชนิด

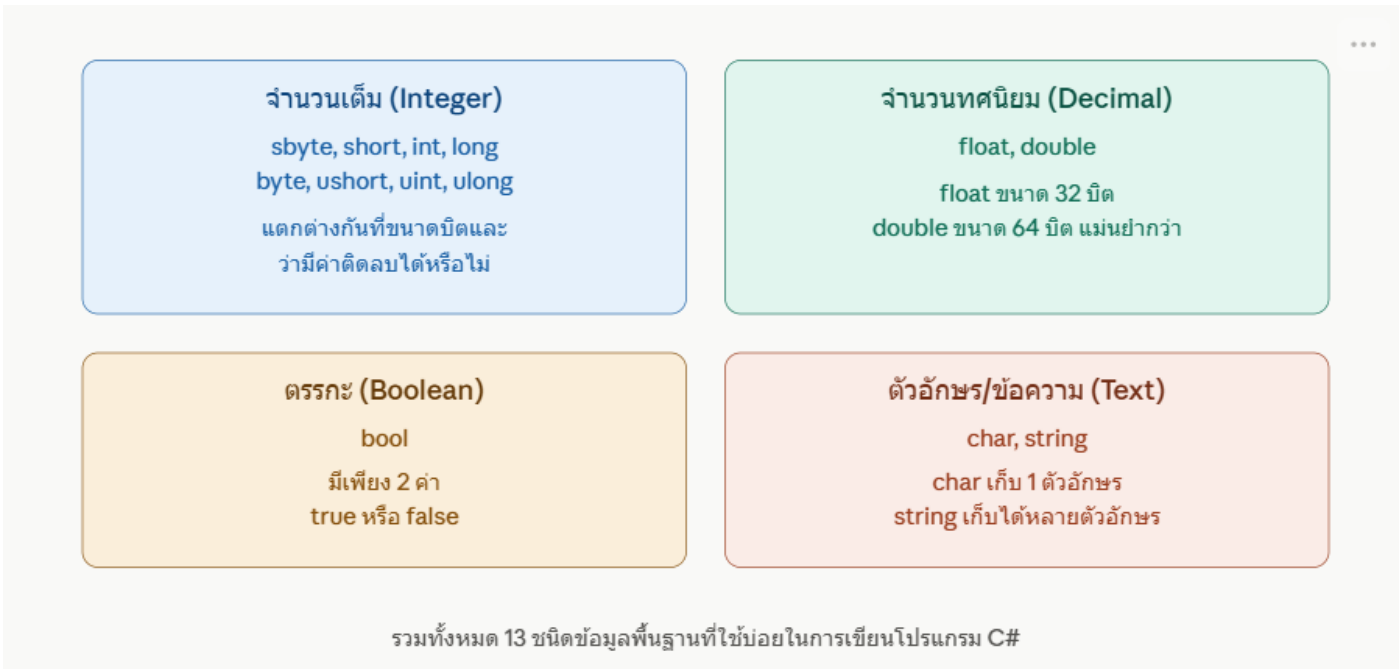
ชนิดข้อมูล	คำอธิบาย	ขอบเขตข้อมูล
sbyte	จำนวนเต็มทั้งบวกและลบ ขนาด 8 บิต	-128 ถึง 127
short	จำนวนเต็มทั้งบวกและลบ ขนาด 16 บิต	-32,768 ถึง 32,767
int	จำนวนเต็มทั้งบวกและลบ ขนาด 32 บิต	-2,147,483,648 ถึง 2,147,483,647
long	จำนวนเต็มทั้งบวกและลบ ขนาด 64 บิต	-2^{63} ถึง $2^{63}-1$
byte	จำนวนเต็มบวก ขนาด 8 บิต	0-255
ushort	จำนวนเต็มบวก ขนาด 16 บิต	0-65,535
uint	จำนวนเต็มบวก ขนาด 32 บิต	0-4,294,967,295
ulong	จำนวนเต็มบวก ขนาด 64 บิต	0- $2^{64}-1$
float	จำนวนทศนิยม ขนาด 32 บิต	ค่าประมาณ $\pm 1.4 \times 10^{-45}$ ถึง $\pm 3.4 \times 10^8$
double	จำนวนทศนิยม ขนาด 64 บิต	ค่าประมาณ $\pm 4.9 \times 10^{-324}$ ถึง $\pm 1.8 \times 10^{308}$
bool	ข้อมูลชนิดตรรกะ	มี 2 ค่า คือ true และ false
char	ข้อมูลชนิดตัวอักษรตัวเดียว	เช่น 'A', '1'
string	ข้อมูลที่ประกอบด้วยตัวอักษรหลายตัว	เช่น "Hello"

ตัวอย่างการใช้งานชนิดข้อมูลในสถานการณ์จริง

csharp

```
sbyte temperature = -15;    // อุณหภูมิติดลบได้ ใช้ sbyte
int studentCount = 450;     // จำนวนนักเรียนทั้งโรงเรียน
long population = 70000000; // ประชากรประเทศ ตัวเลขใหญ่มาก
double gpa = 3.75;         // เกรดเฉลี่ยมีทศนิยม
bool isPassed = true;      // สถานะผ่าน/ไม่ผ่าน
char grade = 'A';          // เกรดตัวอักษรเดียว
string fullName = "สมชาย ใจดี"; // ชื่อ-นามสกุล
```

ต่อไปนี้เป็นแผนภาพจำแนกชนิดข้อมูลทั้ง 13 ชนิดตามหมวดหมู่การใช้งาน



ภาพที่ 1 แผนภาพจำแนกชนิดข้อมูลทั้ง 13 ชนิด

3.1.2 ตัวแปร (Variables)

ตัวแปรเป็นการอ้างถึงข้อมูล โดยค่าของตัวแปรสามารถเปลี่ยนแปลงได้ตลอดเวลาที่โปรแกรมกำลังทำงานอยู่ ในภาษาซีชาร์ป ตัวแปรทุกตัวต้องถูกประกาศก่อนที่จะถูกนำมาใช้งาน โดยมีการระบุชนิดข้อมูลที่จะใช้กับตัวแปรนั้น ๆ ไว้เสมอ กฎเกณฑ์การตั้งชื่อตัวแปร

- 1. ตัวอักษรตัวแรกของชื่อต้องเป็นตัวอักษรภาษาอังกฤษ หรือตัวขีดเส้นใต้ (_)
- 2. ชื่อตัวระบุต้องประกอบด้วยตัวอักษรภาษาอังกฤษ (A–Z, a–z) ตัวเลข (0–9) หรือเครื่องหมายขีดเส้นใต้ (_) เท่านั้น
- 3. ชื่อตัวระบุต้องไม่ซ้ำกับคำสงวน (Reserved Word)

ตัวอย่างชื่อตัวแปรที่ต้องและไม่ถูกต้อง

ชื่อตัวแปร	ผล	เหตุผล
studentName	☑ ถูกต้อง	ขึ้นต้นด้วยตัวอักษร ไม่ซ้ำคำสงวน
_score	☑ ถูกต้อง	ขึ้นต้นด้วยขีดเส้นใต้ ใช้ได้
total2	☑ ถูกต้อง	มีตัวเลขอยู่กลาง/ท้ายชื่อได้
2total	✗ ผิด	ขึ้นต้นด้วยตัวเลข
student name	✗ ผิด	มีช่องว่างในชื่อ
int	✗ ผิด	ซ้ำกับคำสงวน

รูปแบบการประกาศตัวแปร

```
ชนิดของข้อมูล ชื่อตัวแปร;
```

```
csharp
double x, y;
// x และ y เป็นชื่อของตัวแปร ที่มีชนิดข้อมูลเป็นจำนวนทศนิยม
```

การเขียนคำอธิบายโปรแกรม (Comment)

การเขียนคำอธิบายมีประโยชน์ในการช่วยให้ผู้เขียนโปรแกรมกลับมาอ่านและเข้าใจโค้ดได้ง่ายขึ้น โดยเฉพาะในโปรแกรมขนาดใหญ่ ทำได้ 2 วิธี

```
csharp

/* วิธีที่ 1: ใช้สำหรับข้อความหลายบรรทัด
   เปิดด้วย /* และปิดด้วย */
   ตัวอย่างเช่นคำอธิบายโปรแกรมายาว ๆ */

int score = 85;    // วิธีที่ 2: ใช้ // นำหน้า สำหรับข้อความบรรทัดเดียว
```

3.1.3 คำสงวน (Reserved Word)

คำสงวนคือคำที่ภาษา C# กำหนดไว้ใช้งานเฉพาะ ไม่สามารถนำไปตั้งเป็นชื่อตัวแปร ชื่อคลาส หรือชื่อค่าคงที่ได้ ตัวอย่างคำสงวนที่พบบ่อย ได้แก่

```
class, public, private, static, void, int, double, string, bool, true, false,
if, else, for, foreach, while, switch, case, return, new, namespace, using,
const, readonly, abstract, virtual, override, interface, try, catch, throw
```

ผู้เรียนควรจดจำคำสงวนที่ใช้บ่อยเหล่านี้ เพื่อหลีกเลี่ยงการนำไปตั้งชื่อตัวแปรโดยไม่ตั้งใจ ซึ่งจะทำให้โปรแกรมเกิดข้อผิดพลาดขณะคอมไพล์

3.1.4 ค่าคงที่ (Constants)

ค่าคงที่เป็นการอ้างถึงข้อมูลเช่นเดียวกับตัวแปร สิ่งที่แตกต่างจากตัวแปรคือ **ค่าของมันไม่สามารถเปลี่ยนแปลงได้อีกหลังจากการประกาศ** ในภาษาซีชาร์ป ค่าคงที่ ต้องถูกประกาศโดยระบุชนิดข้อมูลและค่าตั้งต้นก่อนถูกนำมาใช้งานเสมอ โดยต้องระบุด้วยคีย์เวิร์ด `const`

รูปแบบคำสั่ง

```
const ชนิดของข้อมูล ชื่อค่าคงที่ = ค่าที่ค่าคงที่ถูกใช้เป็นตัวแทน;
```

```
csharp

const int a = 1;
/* a เป็นชื่อของค่าคงที่ ที่มีค่าเท่ากับ 1 มีชนิดข้อมูลเป็นเลขจำนวนเต็ม */

const double PI = 3.14159;
const string SchoolName = "วิทยาลัยเทคนิค";
```

ข้อควรระวัง: หากพยายามเขียนคำสั่งกำหนดค่าใหม่ให้กับค่าคงที่ เช่น `a = 5`; ในภายหลัง ตัวแปลภาษาจะแจ้งข้อผิดพลาดทันทีตั้งแต่ขั้นตอนคอมไพล์

3.1.5 ตัวดำเนินการ 3 ประเภท

① ตัวดำเนินการทางคณิตศาสตร์ (Arithmetic Operator)

ตัวดำเนินการ	คำอธิบาย	ตัวอย่าง
+	บวก	$z = x + y;$
-	ลบ	$z = x - y;$
*	คูณ	$z = x * y;$
/	หาร	$z = x / y;$
%	เศษจากการหาร	$z = x \% y;$

② ตัวดำเนินการเปรียบเทียบเชิงตรรกะ (Logical Operator)

ตัวดำเนินการ	คำอธิบาย	ตัวอย่าง
!	นิเสธ (NOT)	!p เป็นจริงเมื่อ p เป็นเท็จ
&&	และ (AND)	p && q เป็นจริงเมื่อ p และ q เป็นจริงทั้งคู่
	หรือ (OR)	p q เป็นจริงเมื่อ p หรือ q ตัวใดตัวหนึ่งเป็นจริง
^	Exclusive OR	p ^ q เป็นจริงเมื่อ p และ q มีค่าต่างกัน

③ ตัวดำเนินการในการกำหนดค่า (Assignment Operator)

ตัวดำเนินการ	คำอธิบาย	ตัวอย่าง
=	กำหนดค่า	$x = y;$
+=	เพิ่มค่าแล้วกำหนดค่า	$x += y;$ เทียบเท่า $x = x + y;$
-=	ลดค่าแล้วกำหนดค่า	$x -= y;$ เทียบเท่า $x = x - y;$
*=	คูณแล้วกำหนดค่า	$x *= y;$ เทียบเท่า $x = x * y;$
/=	หารแล้วกำหนดค่า	$x /= y;$ เทียบเท่า $x = x / y;$
%=	หาเศษแล้วกำหนดค่า	$x \% = y;$ เทียบเท่า $x = x \% y;$

ตัวดำเนินการอื่น ๆ ที่ควรรู้

- + สำหรับ string คือการนำ string 2 ค่ามาต่อกัน (Concatenate)
- ++ เพิ่มค่า โดย $x++$ เทียบเท่ากับ $x = x + 1$
- -- ลดค่า โดย $x--$ เทียบเท่ากับ $x = x - 1$

ตัวอย่างโปรแกรมรวมตัวดำเนินการทั้ง 3 ประเภท

```
csharp

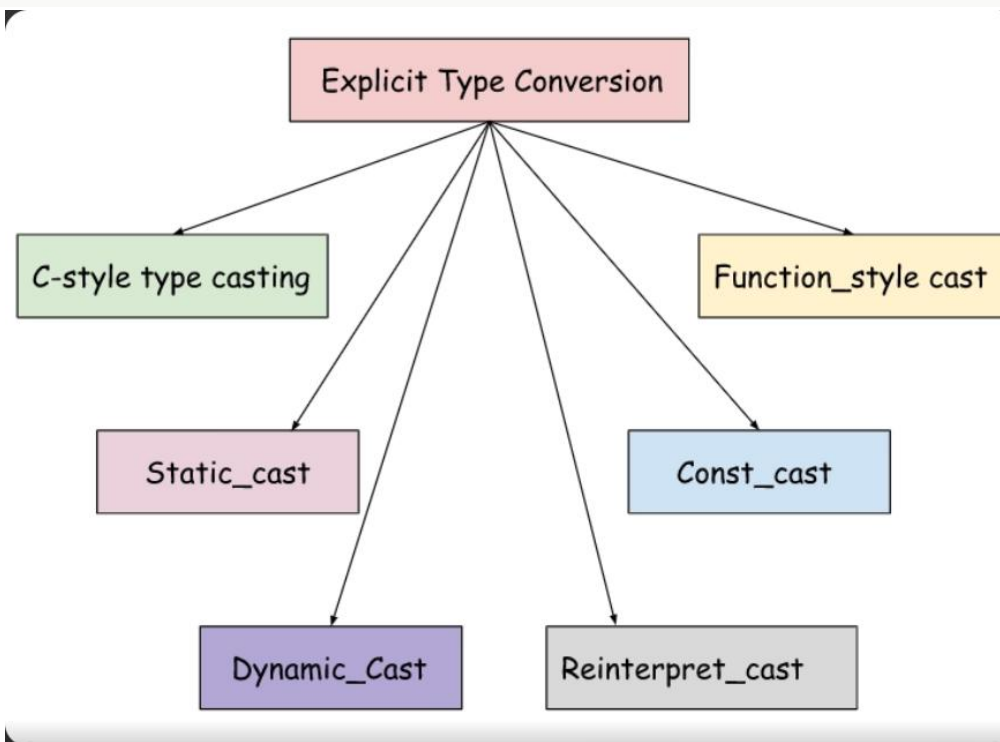
using System;

class Program
{
    static void Main()
    {
        int x = 10, y = 3;
        int sum = x + y;           // ตัวดำเนินการคณิตศาสตร์
        bool result = (x > y) && (y > 0); // ตัวดำเนินการตรรกะ
        x += 5;                    // ตัวดำเนินการกำหนดค่า

        Console.WriteLine("ผลบวก: " + sum);
        Console.WriteLine("ผลตรวจสอบ: " + result);
        Console.WriteLine("ค่า x หลังเพิ่ม: " + x);
    }
}
```

ผลลัพธ์

```
ผลบวก: 13
ผลตรวจสอบ: True
ค่า x หลังเพิ่ม: 15
```



ภาพที่ 2 Explicit Type Conversion

ที่มา https://fity.club/lists/suggestions/dynamic_cast-vs-static_cast/

เนื่องจากภาษาซีชาร์ปเข้มงวดเรื่องชนิดของข้อมูลมาก จึงไม่สามารถนำข้อมูลต่างชนิดกันมาใช้ร่วมกันได้โดยตรง แต่ต้องนำมาทำการแปลงให้เป็นข้อมูลชนิดเดียวกันก่อน จึงจะสามารถนำมาใช้งานร่วมกันได้

เมธอด	ความหมาย	ตัวอย่าง
int.Parse(ข้อความ);	แปลงข้อมูลชนิดตัวอักษร (string) ให้เป็นข้อมูลชนิดตัวเลขจำนวนเต็ม (int)	i = int.Parse(textBox1.Text);
double.Parse(ข้อความ);	แปลงข้อมูลชนิดตัวอักษร (string) ให้เป็นข้อมูลชนิดตัวเลขทศนิยม (double)	i = double.Parse(textBox1.Text);
ToString();	แปลงข้อมูลประเภทใด ๆ ให้เป็นข้อมูลชนิดตัวอักษร (string)	textBox1.Text = a.ToString();
DateTime.Now.ToString();	แปลงข้อมูลชนิดวันเวลา (DateTime) ให้เป็นข้อมูลชนิดตัวอักษร (string)	str1 = DateTime.Now.ToString();

ตัวอย่างสถานการณ์จริงที่ต้องแปลงชนิดข้อมูล

```
csharp
using System;

class Program
{
    static void Main()
    {
        Console.Write("กรุณาป้อนอายุ: ");
        string input = Console.ReadLine();    // รับค่าเป็น string เสมอ
        int age = int.Parse(input);           // แปลงเป็น int เพื่อนำไปคำนวณ

        int nextYearAge = age + 1;
        Console.WriteLine("ปีหน้าคุณจะมีอายุ " + nextYearAge + " ปี");

        int score = 95;
        string scoreText = score.ToString();  // แปลง int กลับเป็น string เพื่อแสดงผล
        Console.WriteLine("คะแนน: " + scoreText);
    }
}
```

คำอธิบาย: ค่าที่รับจากผู้ใช้งานผ่าน Console.ReadLine() หรือ TextBox จะมีชนิดข้อมูลเป็น string เสมอ หากต้องการนำไปคำนวณทางคณิตศาสตร์ จำเป็นต้องแปลงเป็นชนิดตัวเลขก่อนด้วย int.Parse() หรือ double.Parse()

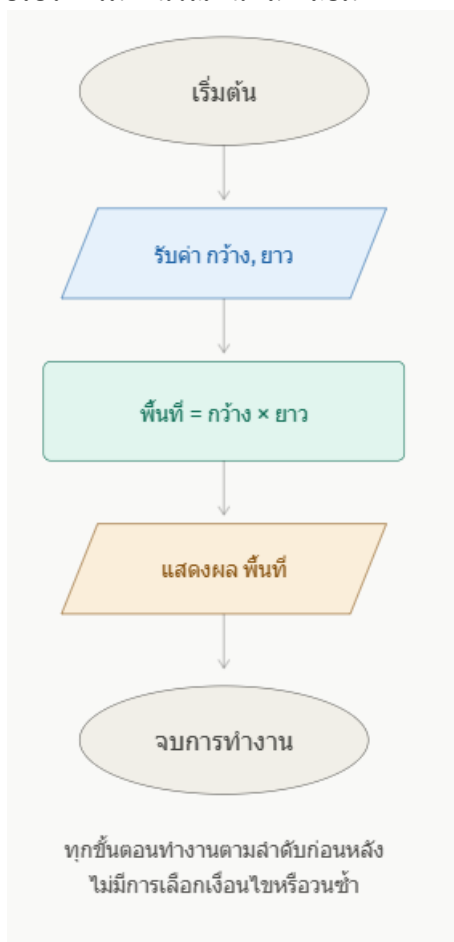
3.1.7 การเขียนโปรแกรมแบบลำดับ (Sequence)

การเขียนโปรแกรมแบบโครงสร้างเป็นการเขียนโปรแกรมที่มีรูปแบบชัดเจน ไม่ซ้ำซ้อน แบ่งออกเป็น 3 รูปแบบ คือ แบบลำดับ แบบเลือก และแบบวนซ้ำ ในหัวข้อนี้จะกล่าวถึงการเขียนโปรแกรมแบบลำดับ ซึ่งมีลักษณะการทำงานตามลำดับก่อนหลังของคำสั่งที่เขียนไว้

สัญลักษณ์ผังงานที่ใช้ในการเขียนผังงานแบบลำดับ

สัญลักษณ์	รูปร่าง	ความหมาย
วงรี (Oval)	○	เริ่มต้น หรือ จบการทำงาน
สี่เหลี่ยมผืนผ้า (Rectangle)	□	กำหนดค่า/ประมวลผล เช่น BackColor = Color.Yellow;
สี่เหลี่ยมด้านขนาน (Parallelogram)	▱	รับค่าหรือแสดงผล เช่น MessageBox.Show("Hello");
ลูกศร (Arrow)	→	แสดงทิศทางการเชื่อมโยงของลำดับ

ต่อไปนี้เป็นตัวอย่างผังงานแบบลำดับสำหรับโปรแกรมคำนวณพื้นที่สี่เหลี่ยม



ภาพที่ 3 ผังงานแบบลำดับ

ตัวอย่างโปรแกรมแบบลำดับที่สอดคล้องกับผังงาน

```
csharp
using System;

class Program
{
    static void Main()
    {
        Console.Write("กรุณาป้อนความกว้าง: ");
        double width = double.Parse(Console.ReadLine());

        Console.Write("กรุณาป้อนความยาว: ");
        double length = double.Parse(Console.ReadLine());

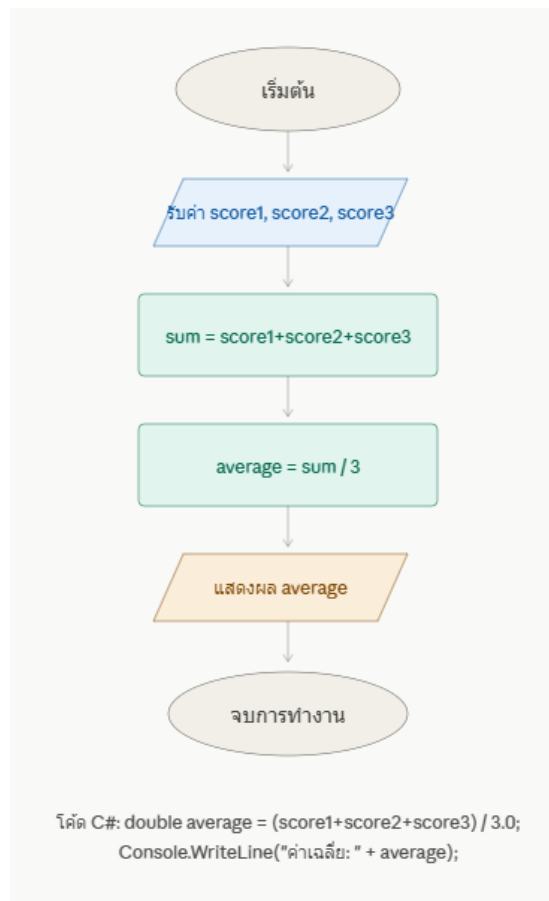
        double area = width * length;

        Console.WriteLine("พื้นที่สี่เหลี่ยมคือ " + area + " ตารางหน่วย");
    }
}
```

ตัวอย่าง

ตัวอย่างที่ 1: โปรแกรมคำนวณค่าเฉลี่ยของคะแนน 3 วิชา

โปรแกรมนี้รับคะแนน 3 วิชา นำมาหาผลรวม แล้วคำนวณค่าเฉลี่ย เป็นตัวอย่างพื้นฐานที่แสดงการรับค่าหลายค่าต่อเนื่องกันแบบลำดับ



```

csharp

using System;

class Program
{
    static void Main()
    {
        Console.Write("กรอกคะแนนวิชาที่ 1: ");
        double score1 = double.Parse(Console.ReadLine());

        Console.Write("กรอกคะแนนวิชาที่ 2: ");
        double score2 = double.Parse(Console.ReadLine());

        Console.Write("กรอกคะแนนวิชาที่ 3: ");
        double score3 = double.Parse(Console.ReadLine());

        double sum = score1 + score2 + score3;
        double average = sum / 3.0;

        Console.WriteLine("ผลรวมคะแนน: " + sum);
        Console.WriteLine("ค่าเฉลี่ย: " + average);
    }
}

```

ผลลัพธ์ตัวอย่าง (สมมติกรอก 80, 75, 90)

```

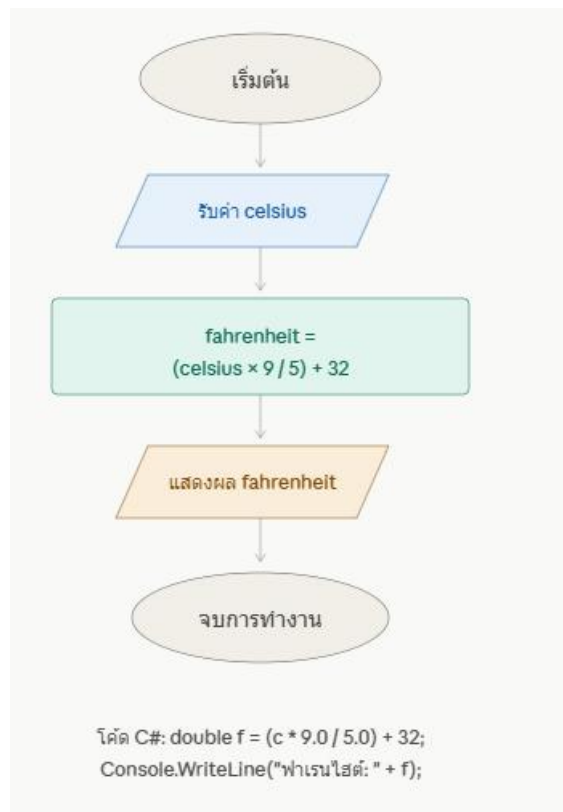
กรอกคะแนนวิชาที่ 1: 80
กรอกคะแนนวิชาที่ 2: 75
กรอกคะแนนวิชาที่ 3: 90
ผลรวมคะแนน: 245
ค่าเฉลี่ย: 81.66666666666667

```



ตัวอย่างที่ 2: โปรแกรมแปลงหน่วยอุณหภูมิ

โปรแกรมนี้นี้แสดงการรับค่าเพียงตัวเดียว แล้วนำไปคำนวณด้วยสูตรทางคณิตศาสตร์ก่อนแสดงผล เป็นตัวอย่างที่แสดงการใช้สูตรคำนวณในขั้นตอนแบบลำดับ



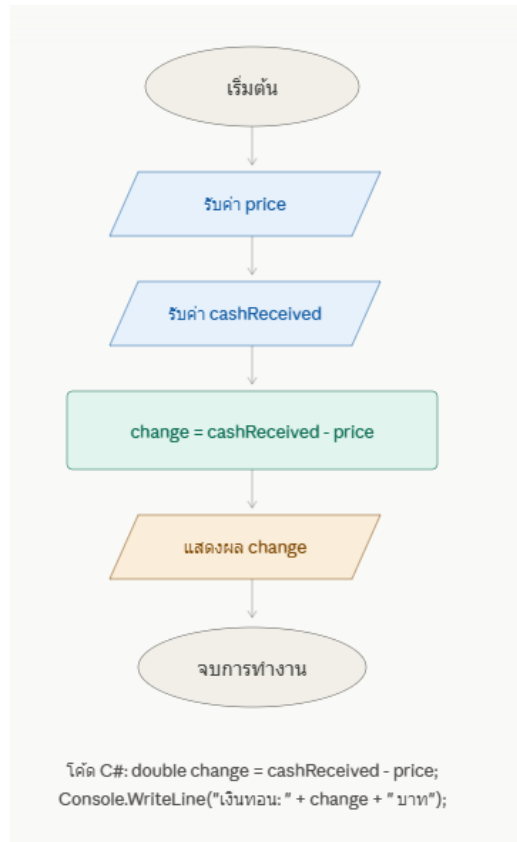
```
using System;
class Program
{
    static void Main()
    {
        Console.WriteLine("กรอกอุณหภูมิเซลเซียส: ");
        double celsius = double.Parse(Console.ReadLine());
        double fahrenheit = (celsius * 9.0 / 5.0) + 32;
        Console.WriteLine(celsius + " องศาเซลเซียส เท่ากับ " + fahrenheit + " องศาฟาเรนไฮต์");
    }
}
```

ผลลัพธ์ตัวอย่าง (สมมติกรอก 37)

กรอกอุณหภูมิเซลเซียส: 37
37 องศาเซลเซียส เท่ากับ 98.6 องศาฟาเรนไฮต์

ตัวอย่างที่ 3: โปรแกรมคำนวณเงินทอนจากการซื้อสินค้า

โปรแกรมนี้นี้แสดงการรับค่า 2 ค่า (ราคาสินค้าและเงินที่รับมา) นำมาคำนวณหาผลต่าง เป็นตัวอย่างที่ใกล้เคียงกับสถานการณ์ในชีวิตประจำวัน เหมาะสำหรับฝึกความเข้าใจการเขียนโปรแกรมแบบลำดับ



```
csharp
using System;

class Program
{
    static void Main()
    {
        Console.Write("ราคาสินค้า: ");
        double price = double.Parse(Console.ReadLine());

        Console.Write("จำนวนเงินที่รับมา: ");
        double cashReceived = double.Parse(Console.ReadLine());

        double change = cashReceived - price;

        Console.WriteLine("เงินทอน: " + change + " บาท");
    }
}
```

ผลลัพธ์ตัวอย่าง (สมมติราคาสินค้า 65 บาท จ่ายมา 100 บาท)

ราคาสินค้า: 65
จำนวนเงินที่รับมา: 100
เงินทอน: 35 บาท

3.1.8 ฟังก์ชัน (Function)

ฟังก์ชัน คือ กลุ่มของคำสั่งที่นำมาเรียงต่อกัน เพื่อการทำงานอย่างใดอย่างหนึ่ง สามารถเรียกใช้ซ้ำ ๆ กันได้ไม่จำกัดจำนวนครั้ง ฟังก์ชันจะช่วยให้เราแบ่งงานออกเป็นงานย่อย ๆ ทำให้โปรแกรมอ่านง่ายและบำรุงรักษาง่ายขึ้น ในหัวข้อนี้จะกล่าวถึงฟังก์ชันสำเร็จรูป 3 กลุ่ม คือ ฟังก์ชันทางคณิตศาสตร์ ฟังก์ชันเกี่ยวกับ String และฟังก์ชันเกี่ยวกับเวลา

① ฟังก์ชันทางคณิตศาสตร์ (Math Class)

เมธอด	ความหมาย	ตัวอย่าง
Math.Pow(ฐาน, เลขชี้กำลัง)	หาค่ายกกำลัง	Math.Pow(x,2) หมายถึงหาค่า x^2
Math.Sqrt(ตัวเลข)	หาค่ารากที่สอง	Math.Sqrt(x) หมายถึงหาค่ารากที่สองของ x
Math.Abs(ตัวเลข)	หาค่า Absolute	Math.Abs(x) หมายถึงหาค่า $ x $
Math.PI	ค่าคงที่ π	มีค่าประมาณ $22 \div 7$
Math.Ceiling(ตัวเลข)	ปัดขึ้น	Math.Ceiling(10.02) = 11
Math.Floor(ตัวเลข)	ปัดลง	Math.Floor(10.99) = 10
Math.Round(ตัวเลข, ตำแหน่ง)	ปัดตามตำแหน่งทศนิยม	Math.Round(23.1243565, 2) = 23.12

```
csharp
double side = 5;
double area = Math.Pow(side, 2);           // ยกกำลังสอง = 25
double diagonal = Math.Sqrt(50);           // รากที่สอง ≈ 7.07
Console.WriteLine("พื้นที่: " + area + " เส้นทแยงมุม: " + Math.Round(diagonal, 2));
```

② ฟังก์ชันเกี่ยวกับ String

เมธอด	ความหมาย	ตัวอย่าง
Length	นับจำนวนอักขระ	len1 = str1.Length;
ToLower()	เปลี่ยนเป็นตัวพิมพ์เล็ก	str2 = str1.ToLower();
ToUpper()	เปลี่ยนเป็นตัวพิมพ์ใหญ่	str2 = str1.ToUpper();
Replace()	แทนที่ข้อความ	str2 = str1.Replace("in","me");
Insert()	แทรกข้อความ	str3 = str1.Insert(3, str2);

```
csharp
string name = "somchai";
Console.WriteLine(name.ToUpper());           // SOMCHAI
Console.WriteLine("ความยาวชื่อ: " + name.Length); // 7
```

③ ฟังก์ชันเกี่ยวกับเวลา

เมธอด	ความหมาย	ตัวอย่าง
DateTime.Now	วันเวลาปัจจุบัน	เช่น 28/07/2569 16:38:10
DateTime.Now.ToShortDateString()	วันที่แบบสั้น	28/07/2569
DateTime.Now.ToLongDateString()	วันที่แบบยาว	28 กรกฎาคม 2569
DateTime.Now.ToShortTimeString()	เวลาแบบสั้น	16:38

csharp

```
Console.WriteLine("วันนี้คือ: " + DateTime.Now.ToLongDateString());
Console.WriteLine("เวลาขณะนี้: " + DateTime.Now.ToShortTimeString());
```

3.2 โครงสร้างภาษา Visual C#

3.2.1 โครงสร้างพื้นฐานของโปรแกรม C#

โปรแกรมภาษา C# ชั้นพื้นฐานที่มีเฉพาะส่วนของโปรแกรมหลัก และไม่มีโปรแกรมน้อยๆ จะมีส่วนประกอบดังนี้

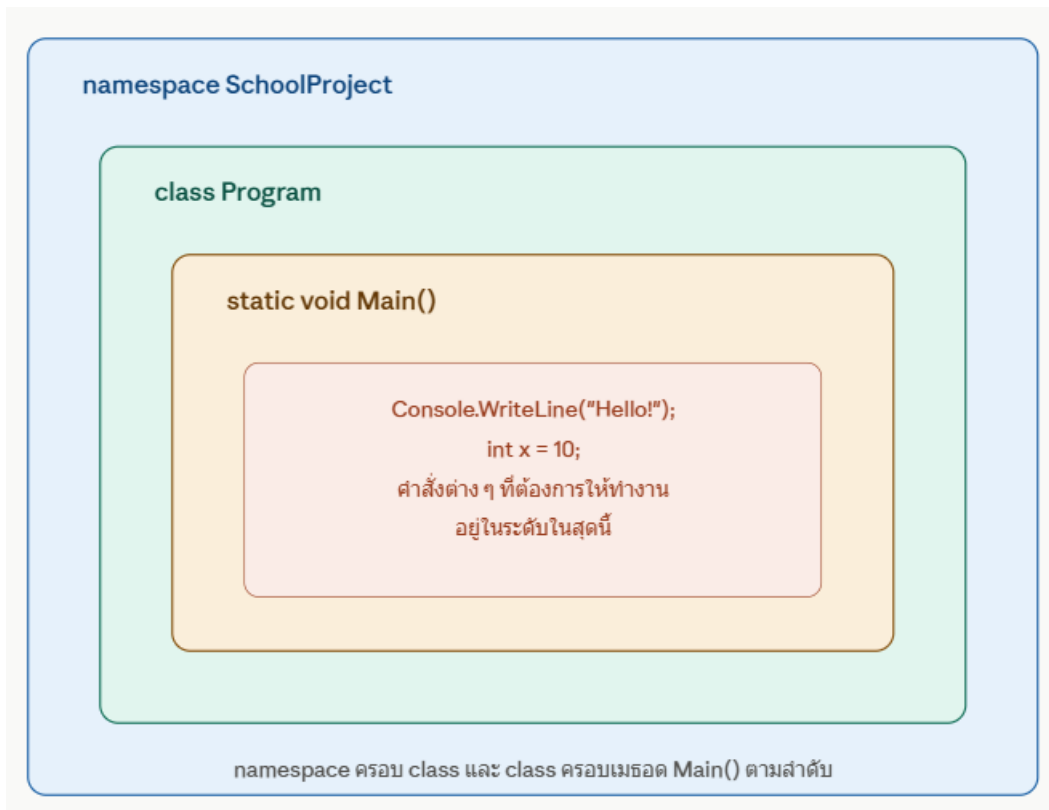
csharp

```
namespace ___(A)___
{
    class ___(B)___
    {
        static void Main()
        {
            ___(C)___
        }
    }
}
```

ตำแหน่ง	ความหมาย
(A)	ระบุชื่อของเนมสเปซ (Namespace)
(B)	ระบุชื่อของคลาส (Class)
(C)	เป็นพื้นที่สำหรับคำสั่งต่าง ๆ ที่ผู้เขียนโปรแกรมต้องการให้คอมพิวเตอร์ปฏิบัติตาม

โครงสร้างนี้เป็นเสมือน "แม่แบบ" ที่ทุกโปรแกรม C# ต้องมีเป็นพื้นฐาน ไม่ว่าโปรแกรมจะมีขนาดเล็กหรือใหญ่เพียงใด ก็ยังคงยึดโครงสร้างนี้เป็นหลักเสมอ

ต่อไปนี้เป็นแผนภาพแสดงโครงสร้างแบบชั้น (Layer) ของโปรแกรม C# ตั้งแต่ระดับนอกสุดไปจนถึงในสุด



ภาพที่ 4 โครงสร้างแบบชั้น (Layer) ของโปรแกรม C#

3.2.2 Namespace

Namespace (เนมสเปซ) คือการจัดกลุ่มของคลาสและองค์ประกอบต่าง ๆ ไว้ภายใต้ชื่อเดียวกัน มีหน้าที่หลักในการป้องกันปัญหาชื่อคลาสซ้ำกัน เมื่อโปรแกรมมีขนาดใหญ่หรือมีการนำไลบรารีภายนอกมาใช้ร่วมกัน ลองจินตนาการว่า Namespace เปรียบเหมือน **ชื่อโรงเรียน** ในขณะที่ Class เปรียบเหมือน **ชื่อห้องเรียน** — โรงเรียน A และโรงเรียน B อาจมีห้อง "ม.3/1" เหมือนกันได้ เพราะอยู่คนละโรงเรียน (namespace) กัน จึงไม่เกิดความสับสน

```
csharp
namespace SchoolProject // ตั้งชื่อเนมสเปซตามโครงการหรือองค์กร
{
    class Student { } // คลาสภายใน namespace นี้
    class Teacher { }
}

namespace LibrarySystem // เนมสเปซอื่น สามารถมีคลาสชื่อ Student เข้าได้
{
    class Student { } // ไม่ขัดแย้งกับ Student ใน SchoolProject
}
```

การเรียกใช้ namespace อื่น ทำได้ด้วยคำสั่ง using ที่ด้านบนสุดของไฟล์ เช่น using System; ซึ่งเป็นการนำเข้าเนมสเปซพื้นฐานที่สุดของ .NET เพื่อให้สามารถใช้คำสั่งอย่าง Console.WriteLine() ได้

3.2.3 Class

Class (คลาส) ในบริบทของโครงสร้างโปรแกรมพื้นฐานนี้ ทำหน้าที่เป็นที่เก็บเมธอดและตัวแปรทั้งหมดของโปรแกรมตามธรรมเนียมปฏิบัติทั่วไป คลาสที่บรรจุเมธอด Main() มักตั้งชื่อว่า Program แต่ผู้เขียนโปรแกรมสามารถตั้งชื่ออื่นได้ตามความเหมาะสม โดยต้องปฏิบัติตามกฎการตั้งชื่อเช่นเดียวกับตัวแปร

```
csharp
```

```
namespace MyApp
{
    class Program    // ชื่อคลาสตามธรรมเนียมปฏิบัติ
    {
        static void Main()
        {
            // คำสั่งการทำงาน
        }
    }
}
```

ข้อสังเกต: ในระยะนี้ผู้เรียนอาจยังไม่จำเป็นต้องเข้าใจความหมายเชิงลึกของคลาสในมุมมอง OOP ทั้งหมด (ซึ่งจะศึกษาอย่างละเอียดในหน่วยถัดไป) เพียงแค่ทราบว่าคุณคือ "ที่บรรจุ" ของโค้ดทั้งหมดที่เขียนขึ้นก็เพียงพอสำหรับหน่วยนี้

3.2.4 เมธอด Main()

static void Main() คือเมธอดที่สำคัญที่สุดของโปรแกรม เรียกว่า **Entry Point** หรือจุดเริ่มต้นการทำงาน ทุกโปรแกรม C# ประเภท Console Application จะต้องมีเมธอดนี้เพียงหนึ่งเดียวเสมอ เมื่อสั่งรันโปรแกรม คอมพิวเตอร์จะเริ่มทำงานจากบรรทัดแรกภายในเมธอด Main() นี้ก่อนเสมอ

ความหมายของแต่ละคำในบรรทัด **static void Main()**

คำ	ความหมาย
static	เมธอดนี้เป็นของคลาสโดยตรง ไม่จำเป็นต้องสร้างวัตถุก่อนจึงจะเรียกใช้ได้
void	เมธอดนี้ไม่มีการส่งค่ากลับ (Return Value)
Main	ชื่อของเมธอด ซึ่ง .NET กำหนดให้ต้องใช้ชื่อนี้เท่านั้นเพื่อเป็นจุดเริ่มต้น
()	วงเล็บสำหรับใส่พารามิเตอร์ (อาจว่างเปล่า หรือมี string[] args เพื่อรับค่าจาก Command-line)

3.2.5 ตัวอย่างโปรแกรมสมบูรณ์พร้อมคำอธิบาย

csharp

```
using System;                                // ①

namespace MyFirstProgram                      // ②
{
    class Program                             // ③
    {
        static void Main()                   // ④
        {
            int score = 85;                    // ⑤
            Console.WriteLine("คะแนนของฉันคือ " + score); // ⑥
        }
    }
}
```

ลำดับ	โค้ด	คำอธิบาย
①	using System;	นำเข้าเนมสเปซพื้นฐานของ .NET เพื่อใช้คำสั่ง Console
②	namespace MyFirstProgram	ตั้งชื่อกลุ่มโปรแกรมว่า MyFirstProgram
③	class Program	ประกาศคลาสหลักชื่อ Program
④	static void Main()	จุดเริ่มต้นการทำงานของโปรแกรม
⑤	int score = 85;	ประกาศตัวแปรและกำหนดค่าเริ่มต้น
⑥	Console.WriteLine(...)	แสดงผลข้อความรวมกับค่าตัวแปรออกทางหน้าจอ

ผลลัพธ์เมื่อรันโปรแกรม

คะแนนของฉันคือ 85

3.2.6 ข้อผิดพลาดที่พบบ่อยเกี่ยวกับโครงสร้างโปรแกรม

ข้อผิดพลาด	สาเหตุ	วิธีแก้ไข
ไม่มีเมธอด Main()	ลืมประกาศจุดเริ่มต้นโปรแกรม	ต้องมีเมธอด static void Main() อยู่ในคลาสใดคลาสหนึ่งเสมอ
ปีกกาเปิด-ปิดไม่ครบ	นับจำนวน { } ผิดพลาด	ตรวจสอบว่าทุก { มี } ปิดคู่กันเสมอ
สะกดผิด main แทน Main	ภาษา C# คำนึงถึงตัวพิมพ์เล็ก-ใหญ่ (Case-Sensitive)	ต้องพิมพ์ Main ขึ้นต้นด้วยตัวพิมพ์ใหญ่เท่านั้น
ลืม using System;	ไม่ได้นำเข้าเนมสเปซที่จำเป็น	เพิ่ม using System; ที่บรรทัดบนสุดของไฟล์

3.3 วิธีติดตั้ง Visual Studio

หมายเหตุสำคัญก่อนเริ่มต้น

เอกสารต้นฉบับของหน่วยนี้อ้างอิงการติดตั้ง Visual Studio 2010 ซึ่งเป็นเวอร์ชันที่เก่ามากแล้วและไม่รองรับ .NET เวอร์ชันปัจจุบัน เนื้อหาฉบับละเอียดนี้จึงปรับปรุงขั้นตอนให้สอดคล้องกับ Visual Studio เวอร์ชันปัจจุบัน (Visual Studio Community) ซึ่งดาวน์โหลดได้ฟรีจากเว็บไซต์ทางการ <https://visualstudio.microsoft.com> โดยหลักการและลำดับขั้นตอนโดยรวมยังคงใกล้เคียงกับเวอร์ชันเดิม

3.3.1 ขั้นตอนการดาวน์โหลดตัวติดตั้งโปรแกรม

1. เข้าเว็บไซต์ <https://visualstudio.microsoft.com>
2. เลือกดาวน์โหลดรุ่น Community ซึ่งเป็นรุ่นที่ใช้งานได้ฟรีสำหรับนักเรียน นักศึกษา และผู้พัฒนารายบุคคล
3. รอให้ไฟล์ตัวติดตั้ง (Installer) ขนาดเล็กดาวน์โหลดเสร็จ แล้วดับเบิลคลิกเพื่อเริ่มการติดตั้ง

3.3.2 ขั้นตอนการติดตั้งทีละขั้นตอน

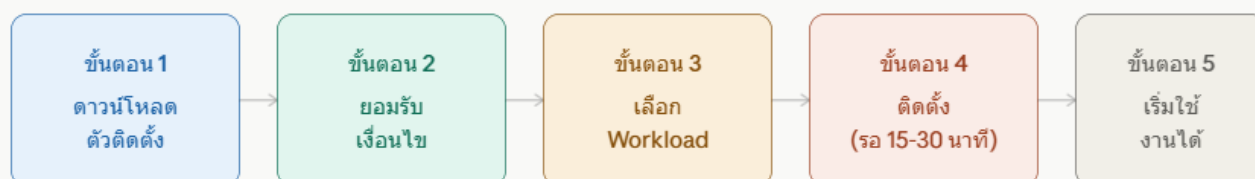
ขั้นตอนที่ 1: เริ่มต้นตัวติดตั้ง

เมื่อเปิดไฟล์ตัวติดตั้ง โปรแกรมจะแสดงข้อความต้อนรับและเริ่มดาวน์โหลดองค์ประกอบพื้นฐานที่จำเป็นสำหรับตัวติดตั้งเองก่อน จากนั้นจะปรากฏหน้าจอให้ยอมรับเงื่อนไขการใช้งาน (License Terms) และนโยบายความเป็นส่วนตัว ให้กด Continue เพื่อดำเนินการต่อ

ขั้นตอนที่ 2: เลือกชุดเครื่องมือ (Workloads)

นี่คือขั้นตอนสำคัญที่สุด เนื่องจาก Visual Studio เวอร์ชันใหม่อนุญาตให้เลือกติดตั้งเฉพาะชุดเครื่องมือ (Workload) ที่ต้องการใช้งาน แทนที่จะติดตั้งทุกอย่างแบบ Full เหมือนเวอร์ชันเก่า ซึ่งช่วยประหยัดพื้นที่ดิสก์และเวลาในการติดตั้ง สำหรับการเรียนเขียนโปรแกรม C# ให้เลือกชุดเครื่องมือ ".NET desktop development" ซึ่งจะติดตั้งเครื่องมือที่จำเป็นสำหรับสร้างโปรแกรม Console Application และ Windows Forms Application ด้วยภาษา C#

ต่อไปนี้เป็นแผนภาพสรุปขั้นตอนการติดตั้งทั้งหมด



เลือกเฉพาะ Workload ".NET desktop development" สำหรับเขียนโปรแกรม C#

ภาพที่ 5 แผนภาพสรุปขั้นตอนการติดตั้ง

ขั้นตอนที่ 3: เริ่มการติดตั้ง

หลังจากเลือกชุดเครื่องมือที่ต้องการแล้ว ให้กดปุ่ม Install มุมล่างขวาของหน้าจอ ตัวติดตั้งจะเริ่มดาวน์โหลดและติดตั้งองค์ประกอบทั้งหมดโดยอัตโนมัติ ใช้เวลาประมาณ 15-30 นาที ขึ้นอยู่กับความเร็วอินเทอร์เน็ตและสเปกเครื่องคอมพิวเตอร์

ขั้นตอนที่ 4: รอการติดตั้งเสร็จสมบูรณ์

ระหว่างการติดตั้ง จะมีแถบแสดงความคืบหน้า (Progress Bar) ให้ผู้เรียนรอจนกว่าแถบจะเต็ม โดยไม่ควรปิดโปรแกรมหรือตัดการเชื่อมต่ออินเทอร์เน็ตระหว่างขั้นตอนนี้

ขั้นตอนที่ 5: เริ่มต้นใช้งานครั้งแรก

เมื่อการติดตั้งเสร็จสมบูรณ์ โปรแกรมจะเปิดขึ้นมาโดยอัตโนมัติ และอาจมีขั้นตอนให้ลงชื่อเข้าใช้ด้วยบัญชี Microsoft (สามารถกด "Skip for now" ข้ามได้หากยังไม่มีบัญชี) จากนั้นให้เลือกธีมสีที่ต้องการ (Dark/Light/Blue) แล้วกด Start Visual Studio เพื่อเริ่มใช้งาน

3.3.3 ตารางสรุปขั้นตอนการติดตั้งแบบย่อ

ขั้นตอน	การกระทำ	หมายเหตุ
---------	----------	----------

1	ดาวน์โหลดตัวติดตั้งจาก visualstudio.microsoft.com	เลือกรุ่น Community (ฟรี)
2	เปิดไฟล์ติดตั้งและกด Continue	ยอมรับเงื่อนไขการใช้งาน
3	เลือก Workload ".NET desktop development"	จำเป็นสำหรับเขียน C#
4	กด Install แล้วรอ	ใช้เวลา 15-30 นาที
5	เริ่มต้นใช้งานโปรแกรม	เลือกธีมและเริ่มเขียนโค้ดได้ทันที

3.3.4 ข้อกำหนดเบื้องต้นของเครื่องคอมพิวเตอร์ (System Requirements)

รายการ	ข้อกำหนดขั้นต่ำ
ระบบปฏิบัติการ	Windows 10 เวอร์ชัน 1909 ขึ้นไป หรือ Windows 11
หน่วยความจำ (RAM)	อย่างน้อย 4 GB (แนะนำ 8 GB ขึ้นไป)
พื้นที่ว่างในดิสก์	อย่างน้อย 20-50 GB (ขึ้นอยู่กับ Workload ที่เลือก)
การเชื่อมต่ออินเทอร์เน็ต	จำเป็นสำหรับการดาวน์โหลดและติดตั้ง

หมายเหตุ: ผู้เรียนที่ใช้เครื่องคอมพิวเตอร์ macOS หรือ Linux สามารถใช้ Visual Studio Code ร่วมกับส่วนขยาย "C# Dev Kit" แทนได้ ซึ่งใช้ทรัพยากรเครื่องน้อยกว่าและติดตั้งได้รวดเร็วกว่า

3.3.5 ปัญหาที่พบบ่อยขณะติดตั้งและวิธีแก้ไข

ปัญหา	สาเหตุที่เป็นไปได้	วิธีแก้ไข
ติดตั้งช้ามาก หรือค้าง	อินเทอร์เน็ตไม่เสถียร	ตรวจสอบสัญญาณอินเทอร์เน็ต ลองปิด-เปิดตัวติดตั้งใหม่
พื้นที่ดิสก์ไม่พอ	เลือก Workload มากเกินความจำเป็น	ยกเลิกการเลือก Workload ที่ไม่ใช้งาน เลือกเฉพาะที่จำเป็น
ติดตั้งเสร็จแต่เปิดโปรแกรมไม่ได้	ไฟล์ติดตั้งไม่สมบูรณ์	ถอนการติดตั้ง (Uninstall) แล้วดาวน์โหลดใหม่อีกครั้ง
ขึ้นข้อความ "Windows เวอร์ชันไม่รองรับ"	ระบบปฏิบัติการเก่าเกินไป	อัปเดต Windows หรือใช้ Visual Studio Code แทน

3.4 การใช้โปรแกรม Visual Studio

3.4.1 เริ่มต้นใช้งานและสร้างโปรเจกต์ใหม่

เมื่อเปิดโปรแกรม Visual Studio ขึ้นมาครั้งแรก จะพบหน้าจอเริ่มต้น (Start Window) ให้เลือกดำเนินการต่อไปนี้

- คลิกที่ **Create a new project**
- เลือกประเภทเทมเพลตโปรเจกต์ เช่น **Console App** (สำหรับโปรแกรมที่ทำงานผ่านหน้าจอข้อความ) หรือ **Windows Forms App** (สำหรับโปรแกรมที่มีหน้าต่างกราฟิก) โดยเลือกภาษา **C#** ในตัวกรอง
- ตั้งชื่อโปรเจกต์ (Project Name) ซึ่งก็คือชื่อโปรแกรมคอมพิวเตอร์ที่จะสร้างขึ้น และเลือกตำแหน่งบันทึกไฟล์ (Location)
- คลิก **Create** เพื่อสร้างโปรเจกต์และเข้าสู่หน้าจอหลักของโปรแกรม (IDE)

3.4.2 องค์ประกอบของ IDE (Integrated Development Environment)

ก่อนเริ่มต้นเขียนโปรแกรมคอมพิวเตอร์ด้วยภาษา C# ผู้เรียนควรทำความรู้จักกับเครื่องมือหลักของ Visual Studio เพื่อให้สามารถใช้งานได้อย่างมีประสิทธิภาพ โดย IDE ประกอบด้วยส่วนสำคัญ 7 ส่วน ดังนี้

① เมนูบาร์ (Menu Bar)

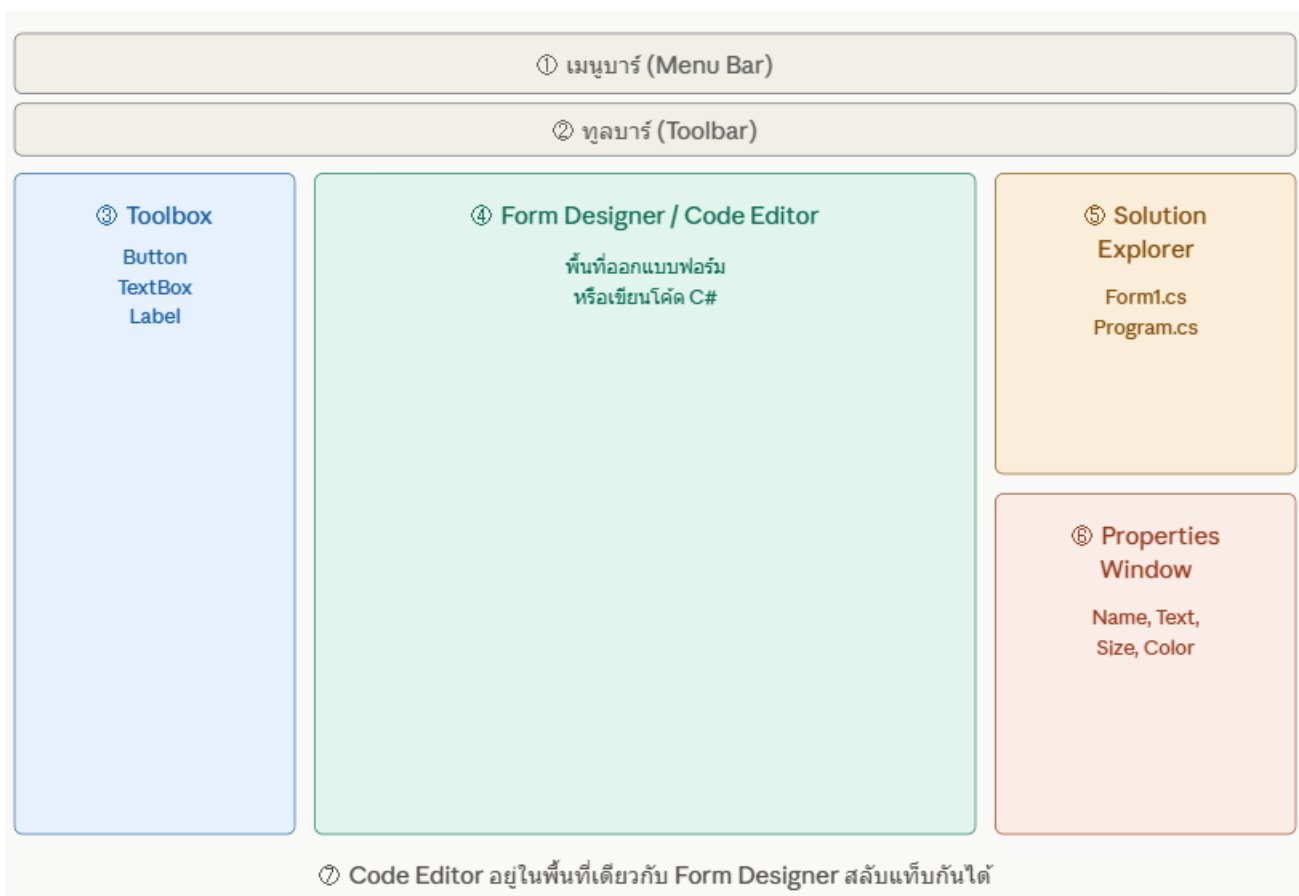
เมนูบาร์เป็นเมนูหลักที่รวบรวมคำสั่งควบคุมการทำงานของ Visual Studio ทั้งหมด โดยจัดเป็นกลุ่มคำสั่งแยกตามประเภทการใช้งาน เช่น **File** (จัดการไฟล์และโปรเจกต์), **Edit** (แก้ไขโค้ด), **View** (เปิด/ปิดหน้าต่างเครื่องมือ), **Build** (คอมไพล์โปรแกรม), **Debug** (ดีบั๊กและรันโปรแกรม)

② ทูลบาร์ (Toolbar)

ทูลบาร์เป็นการรวบรวมคำสั่งในเมนูบาร์บางคำสั่งที่มีการใช้งานบ่อย ๆ มาแสดงไว้เป็นปุ่มไอคอน เพื่อให้สามารถเรียกใช้งานได้สะดวกขึ้นในคลิกเดียว เช่น ปุ่ม  **Start** (รันโปรแกรม) ปุ่ม  **Save** (บันทึกไฟล์) และปุ่มหยุดการทำงาน

③ หน้าต่าง Toolbox

Toolbox เป็นหน้าต่างที่แสดงคอนโทรล (Control) และคอมโพเนนต์ (Component) ต่าง ๆ เพื่อให้สะดวกในการสร้างโปรแกรม โดยเฉพาะโปรแกรมประเภท Windows Forms Application เช่น ปุ่มกด (Button), กล่องข้อความ (TextBox), ป้ายข้อความ (Label) จะถูกจัดแบ่งเป็นกลุ่ม ๆ ตามลักษณะการใช้งาน เพื่อให้ค้นหาได้ง่ายต่อไปนี้เป็นแผนภาพแสดงตำแหน่งขององค์ประกอบหลักทั้งหมดในหน้าจอ IDE



ภาพที่ 6 แสดงตำแหน่งขององค์ประกอบหลัก

④ หน้าต่าง Form Designer

Form Designer เป็นหน้าต่างที่ใช้สำหรับการออกแบบหน้าตาของโปรแกรม (User Interface) โดยการลากคอนโทรลต่าง ๆ จาก Toolbox มาวางบน Form ตามต้องการ เช่น การลากปุ่ม Button มาวางแล้วปรับตำแหน่งและขนาดตามที่ต้องการ เหมาะสำหรับการสร้างโปรแกรมประเภท Windows Forms Application ที่มีหน้าจอกกราฟิกโต้ตอบกับผู้ใช้

⑤ หน้าต่าง Solution Explorer

Solution Explorer เป็นหน้าต่างแสดงรายการของไอเท็ม (Item) ทั้งหมดที่มีอยู่ในโปรเจกต์ เช่น ไฟล์ฟอร์ม (Form), โมดูล (Module), คอมโพเนนต์ (Component) และคลาส (Class) เปรียบเสมือน "สารบัญ" ของโปรเจกต์ทั้งหมด ช่วยให้ผู้ใช้สามารถสลับไปมาระหว่างไฟล์ต่าง ๆ ได้อย่างเป็นระบบ โดยเฉพาะเมื่อโปรเจกต์มีหลายไฟล์

⑥ หน้าต่าง Properties Window

Properties Window เป็นหน้าต่างที่ใช้แสดงและกำหนดคุณสมบัติ (Property) ของชิ้นส่วนหรือองค์ประกอบต่าง ๆ ของโปรแกรมที่กำลังจะสร้างขึ้น เช่น เมื่อคลิกเลือกปุ่ม Button บน Form หน้าต่างนี้จะแสดงคุณสมบัติต่าง ๆ ของปุ่มนั้น เช่น Name (ชื่ออ้างอิงในโค้ด), Text (ข้อความบนปุ่ม), Size (ขนาด), BackColor (สีพื้นหลัง) ซึ่งสามารถปรับแต่งได้โดยไม่ต้องเขียนโค้ด

⑦ หน้าต่าง Code Editor

Code Editor เป็นหน้าต่างที่ใช้สำหรับเขียนคำสั่งในภาษา C# เพื่อกำหนดการทำงานของโปรแกรมตามที่ต้องการ มีความสามารถช่วยเหลือผู้เขียนโปรแกรม เช่น IntelliSense ที่แนะนำคำสั่งขณะพิมพ์ และการตรวจสอบไวยากรณ์แบบเรียลไทม์ (ขีดเส้นสีแดงใต้จุดที่ผิดพลาด)

3.4.3 ตารางสรุปองค์ประกอบของ IDE

องค์ประกอบ	หน้าที่หลัก	ใช้บ่อยเมื่อ...
เมนูบาร์ (Menu Bar)	รวบรวมคำสั่งควบคุมการทำงานทั้งหมด	ต้องการคำสั่งที่ไม่มีในทูลบาร์
ทูลบาร์ (Toolbar)	ปุ่มลัดคำสั่งที่ใช้บ่อย	รันโปรแกรม บันทึกไฟล์
Toolbox	แสดงคอนโทรลสำหรับออกแบบหน้าจอ	สร้าง Windows Forms Application
Form Designer	ออกแบบหน้าต่างโปรแกรมแบบลากวาง	จัดวาง Button, TextBox, Label
Solution Explorer	แสดงรายการไฟล์ทั้งหมดในโปรเจกต์	สลับไปมาระหว่างไฟล์
Properties Window	ปรับแต่งคุณสมบัติของคอนโทรล	ตั้งชื่อ สี ขนาด ข้อความ
Code Editor	เขียนคำสั่งภาษา C#	เขียนตรรกะการทำงานของโปรแกรม

3.4.4 ตัวอย่างขั้นตอนการใช้งาน IDE สร้างโปรแกรมง่าย ๆ

เพื่อให้เห็นภาพการทำงานร่วมกันขององค์ประกอบทั้งหมด ขอยกตัวอย่างการสร้างโปรแกรม Windows Forms Application แบบง่าย ที่มีปุ่มกดแล้วแสดงข้อความทักทาย

- 1. สร้างโปรเจกต์ใหม่ประเภท Windows Forms App
- 2. ใน Toolbox ลากคอนโทรล Button มาวางบน Form Designer
- 3. คลิกเลือกปุ่มที่วางไว้ แล้วไปที่ Properties Window เปลี่ยนคุณสมบัติ Text เป็น "ทักทาย"
- 4. ดับเบิลคลิกที่ปุ่มบน Form Designer เพื่อเปิด Code Editor โดยอัตโนมัติ พร้อมสร้างเหตุการณ์ (Event) การคลิกปุ่มให้
- 5. เขียนโค้ดในส่วนที่เปิดขึ้นมา ดังนี้

csharp

```
private void button1_Click(object sender, EventArgs e)
{
    MessageBox.Show("สวัสดี นักเรียน ปวช.3");
}
```

- 6. กดปุ่ม Start บนทูลบาร์ (หรือกด F5) เพื่อรันโปรแกรมทดสอบ
- 7. เมื่อคลิกปุ่ม "ทักทาย" บนหน้าต่างโปรแกรมที่เปิดขึ้นมา จะปรากฏกล่องข้อความแสดงคำว่า "สวัสดี นักเรียน ปวช.3"

คำอธิบาย: ตัวอย่างนี้แสดงให้เห็นว่า Toolbox, Form Designer, Properties Window และ Code Editor ทำงานประสานกันอย่างไรในการสร้างโปรแกรมหนึ่งโปรแกรม ซึ่งเป็นกระบวนการพื้นฐานที่ผู้เรียนจะต้องใช้ซ้ำ ๆ ตลอดการเรียนวิชานี้

3.4.5 ข้อผิดพลาดที่พบบ่อยเมื่อใช้งาน IDE

ปัญหา	สาเหตุ	วิธีแก้ไข
มองไม่เห็นหน้าต่าง Toolbox/Properties	หน้าต่างถูกปิดไปโดยไม่ตั้งใจ	ไปที่เมนู View แล้วเลือกเปิดหน้าต่างนั้นกลับมา
กด Start แล้วโปรแกรมไม่รัน	มีข้อผิดพลาดในโค้ด (Error)	ดูข้อความ Error ที่หน้าต่าง Error List แล้วแก้ไขตามจุดที่ระบุ
ลากคอนโทรลแล้วหาไม่เจอ	วางคอนโทรลไว้นอกขอบเขต Form	ใช้ Solution Explorer เปิด Form อีกครั้งแล้ว เลื่อนหาคอนโทรล
แก้ไข Properties แล้วไม่มีผล	แก้ไขผิดคอนโทรล	ตรวจสอบว่าได้คลิกเลือกคอนโทรลที่ต้องการแก้ไข จริงก่อนปรับค่า